

Software Specification Document

Nepal Ebilling Software

1. Introduction

Nepal Ebilling Software is an advanced billing solution designed to streamline and automate the billing processes for businesses in Nepal. It is built on a modern tech stack including Next.js, Python Django, and FastAPI, with DataHub server integration for efficient data management and analytics.

2. Purpose

The purpose of this document is to provide a detailed specification for the Nepal Ebilling Software, outlining the functional and non-functional requirements, system architecture, user roles, and technology stack. This document serves as a guideline for developers, testers, and stakeholders involved in the project.

3. System Overview

Nepal Ebilling Software aims to provide a seamless and user-friendly billing experience, featuring real-time data processing, robust security measures, and comprehensive reporting capabilities. The software is designed to support multiple user roles, ensuring that different types of users can efficiently manage billing operations.

4. Technology Stack

- **Frontend:** Next.js
- **Backend:** Python Django, FastAPI
- **Data Management:** DataHub server
- **Database:** PostgreSQL
- **Authentication:** JWT (JSON Web Tokens)
- **Deployment:** Docker
- **CI/CD:** GitHub Actions

5. Functional Requirements

5.1 User Authentication and Authorization

- Users must be able to register and log in to the system.

- Different user roles (Admin, Manager, Accountant, Customer) must have specific permissions.
- Authentication should be handled using JWT tokens.

5.2 Billing Management

- Users must be able to create invoices.
- Invoices should be exportable in PDF formats.

5.3 Customer Management

- Users must be able to add, edit, and delete customer information.
- Customer data should include name, contact details, billing address, and payment history.

5.4 Payment Processing

- Users should be able to track payment status (paid, pending, overdue).

5.5 Reporting and Analytics

- The system should provide comprehensive reports on Sales, Sales Return, Purchase, Purchase Returns, payments, and customer data.
- Users must be able to generate custom reports based on various filters (date range, invoice status).
- Integration with DataHub server for advanced data analytics and visualization.

6. Non-Functional Requirements

6.1 Performance

- The system should handle up to 1000 concurrent users.
- Response time for critical operations should be less than 2 seconds.

6.2 Usability

- The user interface should be intuitive and easy to navigate.
- Comprehensive user documentation and help resources should be provided.

6.3 Scalability

- The system architecture should support horizontal scaling.

6.4 Reliability

- The system should have an uptime of 99.9%.
- Automated backups should be scheduled regularly.

7. System Architecture

The system architecture is based on a microservices approach, ensuring modularity and scalability.

- **Frontend:** Next.js application hosted on a CDN for fast delivery.
- **Backend:** Python Django for core business logic, FastAPI for handling asynchronous operations.
- **Database:** PostgreSQL for relational data storage.
- **DataHub Server:** For data integration, analytics, and reporting.
- **Deployment:** Containerized using Docker, orchestrated with Kubernetes.

8. User Roles and Permissions

- **Admin:** Full access to all features and settings.

Other user also can be created where custom permission could be provided according to roles.

9. Development and Deployment

- **Version Control:** Git
- **Continuous Integration:** GitHub Actions for automated testing and deployment.
- **Containerization:** Docker for consistent environments across development and production.
- **Orchestration:** Kubernetes for managing containerized applications.
- **Monitoring:** Prometheus and Grafana for monitoring system performance and health.

10. Testing

- **Unit Testing:** For individual components and services.
- **Integration Testing:** To ensure all services work together as expected.
- **Performance Testing:** To validate the system's performance under load.
- **Security Testing:** Regular security assessments and penetration testing.

11. Maintenance and Support

- Regular updates and patches for security and performance improvements.
- 24/7 support for critical issues.
- Detailed documentation and user manuals.

12. Conclusion

This software specification document provides a comprehensive overview of the Nepal Ebilling Software, outlining the functional and non-functional requirements, system architecture,

technology stack, and user roles. It serves as a roadmap for the development and deployment of a robust and scalable billing solution tailored to the needs of businesses in Nepal.